

Policy Gradient Reinforcement Learning on CartPole

A Fundamental Study Note for Implementation

1 Goal of This Note

We want to understand how to train a neural network policy on the CartPole environment. CartPole is a classic reinforcement learning example where a cart moves left or right in order to keep a pole balanced.

The important idea is:

$$\boxed{\text{state} \rightarrow \text{neural network policy} \rightarrow \text{action}}$$

The neural network receives the current state of the environment and outputs a probability distribution over actions. Then we sample or choose an action, apply it in the environment, receive reward, and use policy gradient to make good actions more likely in the future.

2 Prerequisite Concepts

Before implementing CartPole with policy gradient, we need a few concepts.

2.1 State

A state is the information the environment gives to the agent at a time step. For CartPole, the state is a vector with four real numbers:

$$s_t = \begin{bmatrix} x_t \\ \dot{x}_t \\ \theta_t \\ \dot{\theta}_t \end{bmatrix}$$

where:

- x_t is the cart position;
- $\dot{x}_t$ is the cart velocity;
- θ_t is the pole angle;
- $\dot{\theta}_t$ is the pole angular velocity.

So CartPole gives the agent a four-dimensional observation:

$$\boxed{s_t \in \mathbb{R}^4}$$

2.2 Action

An action is what the agent chooses to do. In CartPole, there are two possible actions:

$$a_t \in \{0, 1\}$$

Usually:

- $a_t = 0$: push the cart left;
- $a_t = 1$: push the cart right.

So CartPole has a discrete action space with two actions:

$$|\mathcal{A}| = 2$$

2.3 Reward

At each step, the environment returns a reward. In CartPole, the agent usually receives reward +1 for every time step that the pole remains balanced.

So if the pole stays balanced for 200 steps, the total reward is approximately:

$$R_{\text{episode}} = 200$$

The agent's goal is to maximize total future reward.

2.4 Episode

An episode is one full run of the environment. For CartPole, an episode starts with the pole almost upright and ends when the pole falls too much, the cart moves too far, or the maximum time limit is reached.

A trajectory, or rollout, is the sequence:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$$

This is the data generated by one episode.

3 The Policy

The policy is the agent's behavior rule. It tells us which action the agent should take in each state.

In stochastic reinforcement learning, the policy is written as:

$$\pi_{\theta}(a_t | s_t)$$

This means:

$$\text{probability of taking action } a_t \text{ given state } s_t$$

The parameter θ represents the weights of the neural network.

3.1 Neural Network as Policy

For CartPole, the neural network receives four inputs and outputs two logits:

$$\mathbb{R}^4 \rightarrow \mathbb{R}^2$$

More explicitly:

$$\begin{bmatrix} x_t \\ \dot{x}_t \\ \theta_t \\ \dot{\theta}_t \end{bmatrix} \xrightarrow{\text{MLP}_\theta} \begin{bmatrix} z_{\text{left}} \\ z_{\text{right}} \end{bmatrix}$$

The values z_{left} and z_{right} are called logits. They are not probabilities yet. They are raw scores. To turn logits into probabilities, we use softmax:

$$\pi_\theta(a = i \mid s_t) = \frac{e^{z_i}}{\sum_{j=1}^2 e^{z_j}}$$

So the network gives something like:

$$\pi_\theta(\text{left} \mid s_t) = 0.35, \quad \pi_\theta(\text{right} \mid s_t) = 0.65$$

Then we can sample an action from this distribution.

4 Interaction Loop

At each time step, the agent does the following:

1. Receive current state s_t from the environment.
2. Pass s_t through the policy neural network.
3. Get action probabilities $\pi_\theta(a_t \mid s_t)$.
4. Sample an action a_t .
5. Apply the action in the environment.
6. Receive reward r_t and next state s_{t+1} .
7. Store the log-probability and reward for training.

Mathematically:

$$s_t \rightarrow \pi_\theta(a_t \mid s_t) \rightarrow a_t \rightarrow r_t, s_{t+1}$$

Implementation-style pseudocode:

```
state = env.reset()
done = False

log_probs = []
rewards = []
```

```

while not done:
    logits = policy_net(state)
    probs = softmax(logits)
    action = sample_from(probs)

    log_prob = log_probability_of_chosen_action(probs, action)

    next_state, reward, done, info = env.step(action)

    log_probs.append(log_prob)
    rewards.append(reward)

    state = next_state

```

The key thing we store is not just the action. We store:

$$\log \pi_{\theta}(a_t \mid s_t)$$

This is needed because policy gradient changes the neural network weights using the log-probability of the sampled action.

5 Return

The return is the total future reward after a time step. For time step t , the discounted return is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

Compactly:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$

where γ is the discount factor.

5.1 Discount Factor

The discount factor γ controls how much the agent cares about future rewards.

$$0 \leq \gamma \leq 1$$

If $\gamma = 0$, the agent only cares about immediate reward. If γ is close to 1, the agent cares a lot about future reward.

For CartPole, common choices are:

$$\gamma = 0.99$$

This means future rewards are very important, which makes sense because the goal is to keep the pole balanced for as long as possible.

6 Objective Function

The policy's goal is to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

This means:

Choose neural network weights θ that produce high-reward trajectories.

The difficulty is that the action is sampled. We cannot directly backpropagate through the environment, because the environment may be non-differentiable. Instead, policy gradient gives us a way to update the policy using sampled experience.

7 Policy Gradient Core Idea

The policy gradient tells us how to change θ to increase expected reward.

The main formula is:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t | s_t) G_t]$$

The intuition is very important:

- If an action was followed by high return, increase its probability.
- If an action was followed by low return, do not reinforce it much.
- If using an advantage estimate, bad actions can be actively pushed down.

So if the robot/cart chooses an action that helps keep the pole balanced, the probability of that action in similar states should increase.

8 Policy Gradient Loss

PyTorch minimizes losses. But reinforcement learning wants to maximize reward. So we use a negative sign.

The REINFORCE-style loss is:

$$\mathcal{L}(\theta) = - \sum_{t=0}^T \log \pi_\theta(a_t | s_t) G_t$$

This is the central implementation formula.

Each chosen action contributes:

$$- \log \pi_\theta(a_t | s_t) G_t$$

If G_t is large, minimizing this loss increases the probability of action a_t in state s_t .

8.1 Why the Negative Sign?

If the return is good, we want to maximize:

$$\log \pi_{\theta}(a_t \mid s_t) G_t$$

But optimizers like Adam usually minimize. So we minimize the negative:

$$-\log \pi_{\theta}(a_t \mid s_t) G_t$$

Thus:

maximize reward-weighted log-probability $\iff$ minimize negative reward-weighted log-probability

9 Simple CartPole Training Algorithm

The basic algorithm is:

1. Initialize policy network π_{θ} .
2. Run one episode using the current policy.
3. Store rewards and log-probabilities.
4. Compute discounted returns G_t .
5. Compute loss:

$$\mathcal{L}(\theta) = - \sum_t \log \pi_{\theta}(a_t \mid s_t) G_t$$

6. Backpropagate the loss.
7. Update θ using an optimizer.
8. Repeat for many episodes.

Implementation-style pseudocode:

```
for episode in range(num_episodes):
    state = env.reset()
    done = False

    log_probs = []
    rewards = []

    while not done:
        logits = policy_net(state)
        dist = Categorical(logits=logits)
        action = dist.sample()

        log_prob = dist.log_prob(action)

        next_state, reward, done, info = env.step(action.item())
```

```

log_probs.append(log_prob)
rewards.append(reward)

state = next_state

returns = compute_discounted_returns(rewards, gamma=0.99)

loss = 0
for log_prob, G in zip(log_probs, returns):
    loss += -log_prob * G

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

10 Why We Sample Instead of Taking the Best Action

During training, we usually sample actions from the policy distribution:

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

This allows exploration.

If the agent always takes the most likely action, it may get stuck doing a bad behavior too early. Sampling lets the policy try different actions and discover better ones.

After training, we may use a deterministic action:

$$a_t = \arg \max_a \pi_{\theta}(a \mid s_t)$$

That means we choose the most likely action.

11 On-Policy Nature of This Algorithm

This basic policy gradient method is on-policy. That means the data used for training comes from the same policy that is being updated.

At episode k , we collect data using the current policy:

$$\pi_{\theta_k}$$

Then we update the same policy using that data:

$$\theta_k \rightarrow \theta_{k+1}$$

After updating, the old data is no longer exactly from the current policy. So simple policy gradient methods usually collect fresh data again.

12 Actor and Critic

The simple REINFORCE algorithm only uses a policy network. But more advanced methods often use two networks:

Network	Formula	Purpose
Actor / Policy	$\pi_{\theta}(a \mid s)$	Chooses actions
Critic / Value	$V_{\phi}(s)$	Estimates how good a state is

The policy is the network we actually use to act. The critic helps training by reducing variance.

12.1 Value Function

The value function estimates expected future reward from a state:

$$V_{\phi}(s_t) \approx \mathbb{E}[G_t \mid s_t]$$

In words:

$$V_{\phi}(s_t) = \text{how good this state is expected to be}$$

12.2 Advantage

Instead of using raw return G_t , we often use advantage:

$$A_t = G_t - V_{\phi}(s_t)$$

Advantage means:

$$\text{Was this action better or worse than expected?}$$

If:

$$A_t > 0$$

then the action was better than expected, so we increase its probability.

If:

$$A_t < 0$$

then the action was worse than expected, so we decrease its probability.

The policy loss becomes:

$$\mathcal{L}_{\text{policy}}(\theta) = - \sum_t \log \pi_{\theta}(a_t \mid s_t) A_t$$

This is closer to what algorithms like A2C and PPO use.

13 CartPole Neural Network Architecture

A simple policy network for CartPole can be an MLP:

$$\mathbb{R}^4 \rightarrow \mathbb{R}^{128} \rightarrow \mathbb{R}^{128} \rightarrow \mathbb{R}^2$$

For example:

$$\begin{aligned}h_1 &= \text{ReLU}(W_1 s + b_1) \\h_2 &= \text{ReLU}(W_2 h_1 + b_2) \\z &= W_3 h_2 + b_3\end{aligned}$$

where:

$$z \in \mathbb{R}^2$$

These two output values are logits for left and right.
Then:

$$\pi_{\theta}(a \mid s) = \text{softmax}(z)$$

14 Implementation Notes

14.1 Use Logits Directly

In PyTorch, it is often better to pass logits directly into a categorical distribution:

```
dist = torch.distributions.Categorical(logits=logits)
action = dist.sample()
log_prob = dist.log_prob(action)
```

This is numerically cleaner than manually doing softmax and log.

14.2 Normalize Returns

For stability, it is common to normalize returns inside each batch:

$$\hat{G}_t = \frac{G_t - \mu_G}{\sigma_G + \epsilon}$$

This does not change the basic idea, but it can make learning more stable.

```
returns = (returns - returns.mean()) / (returns.std() + 1e-8)
```

14.3 Batching Episodes

One episode can be noisy. A better implementation can collect multiple episodes before doing one update.

Instead of:

one episode $\rightarrow$ one update

we can do:

many episodes $\rightarrow$ one update

This gives a better estimate of the policy gradient.

15 Important Intuition

Policy gradient is not supervised learning in the usual sense. There is no label saying:

correct action = right

Instead, the environment only says:

this sequence of actions produced this much reward

Then policy gradient assigns credit to the actions by using:

$$\log \pi_{\theta}(a_t \mid s_t) G_t$$

So the model slowly learns:

In states like this, actions like that tend to produce high future reward.

16 Full Conceptual Summary

For CartPole:

- The environment state has four numbers.
- The action space has two actions: left or right.
- The neural network maps state to two logits.
- Softmax turns logits into action probabilities.
- The agent samples an action from the policy.
- The environment gives reward and next state.
- After an episode, we compute discounted returns.
- We compute the policy gradient loss.
- Backpropagation updates the neural network weights.
- Over many episodes, the policy learns to balance the pole.

The essential formula is:

$$\mathcal{L}(\theta) = - \sum_t \log \pi_{\theta}(a_t \mid s_t) G_t$$

And the essential intuition is:

Actions that led to high reward become more likely.

17 Connection to LLM RL

The same idea applies to language models. For an LLM, the policy is:

$$\pi_{\theta}(\text{next token} \mid \text{previous tokens})$$

For CartPole:

$$\pi_{\theta}(\text{action} \mid \text{state})$$

The structure is very similar:

Setting	State / Context	Action
CartPole	cart position, velocity, pole angle	left or right
LLM	previous tokens / prompt	next token

In both cases, policy gradient increases the probability of actions that lead to higher reward.

18 Minimal Mental Model

The shortest way to remember the whole thing is:

Policy gradient = reward-weighted cross entropy on sampled actions

But unlike supervised learning, the target action was not given by a dataset. The action was sampled by the policy itself, and the reward decides whether that sampled action should become more or less likely.